

Research Paper Question Answering System

Using Retrieval-Augmented Generation (RAG)

A Lab Report Submitted for

Generative AI (24AM812)

Name : Devaki Gopi Krishna Kurakula

Registration No. : 231FA23011

Branch : ACSE-DS

Subject : Generative AI (24AM812)

Faculty : Abdul Rahman

Abstract

This report presents the design and implementation of a Retrieval-Augmented Generation (RAG) based Question Answering system for research papers. The system allows a user to upload a research paper in PDF format and ask natural language questions about its contents. Instead of relying purely on a large language model's internal (and potentially outdated or hallucinated) knowledge, the system retrieves the most semantically relevant sections of the uploaded document and grounds the language model's answer strictly in that retrieved context. The pipeline covers PDF ingestion, text extraction, chunking, embedding generation, semantic retrieval using FAISS, context-grounded answer generation using a local instruction-tuned LLM, and source citation. The system is demonstrated on a sample research paper and automatically answers five standard research-paper analysis questions: objective, methodology, datasets, findings, and limitations.

Contents

1	Introduction	2
2	Problem Statement	2
2.1	Requirements	2
3	System Architecture	3
3.1	Tools and Libraries	3
4	Methodology	3
4.1	PDF Ingestion and Text Extraction	3
4.2	Chunking	3
4.3	Embedding Generation	4
4.4	Vector Store and Retriever	4
4.5	Context-Grounded Generation	4
4.6	Source Citation	5
5	Results	5
6	Viva Voce – Key Discussion Points	5
7	Conclusion	6

1 Introduction

Large Language Models (LLMs) are trained on a fixed corpus up to a certain cutoff date and have no inherent knowledge of a specific, newly uploaded document. Asking an LLM directly about the contents of a research paper it has never seen typically results in either a refusal, a generic answer, or a hallucinated (fabricated) response. Retrieval-Augmented Generation (RAG) solves this problem by combining two components:

1. A **retriever**, which searches the actual document for the most relevant passages given a question.
2. A **generator** (LLM), which produces a natural-language answer using only the retrieved passages as context.

This project implements a complete RAG pipeline that operates entirely offline using free, local components: local sentence embeddings, a FAISS vector index, and a small local instruction-tuned language model.

2 Problem Statement

Build a RAG application that allows users to upload research papers and ask questions about their contents. The system should be able to answer questions such as:

- What is the objective of the paper?
- What methodology was used?
- What datasets were used?
- What are the major findings?
- What are the limitations?

2.1 Requirements

R1. PDF ingestion

R2. Text extraction

R3. Chunking

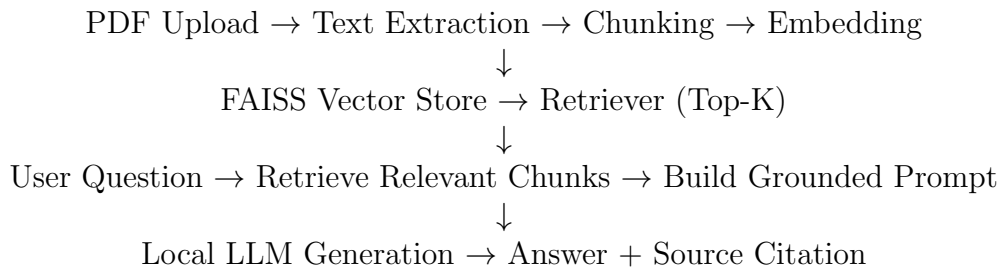
R4. Semantic retrieval

R5. Context-grounded generation

R6. Source citation

3 System Architecture

The pipeline follows the standard RAG architecture, shown conceptually below:



3.1 Tools and Libraries

- **LangChain** – orchestration framework for loaders, splitters, and vector stores
- **PyPDFLoader** – PDF text extraction
- **RecursiveCharacterTextSplitter** – chunking of extracted text
- **HuggingFace sentence-transformers (all-MiniLM-L6-v2)** – local embedding model
- **FAISS** – vector similarity search index
- **Transformers pipeline (Qwen2.5-0.5B-Instruct)** – local generation model

4 Methodology

4.1 PDF Ingestion and Text Extraction

The uploaded PDF is loaded using `PyPDFLoader`, which reads the file page by page and converts it into a list of `LangChain Document` objects. Each `Document` contains the extracted text of one page along with metadata such as the page number.

```
1 loader = PyPDFLoader(str(pdf_file))
2 documents = loader.load()
3 print(f"Loaded {len(documents)} pages")
```

4.2 Chunking

Whole pages are too large and semantically diverse to embed effectively, so each page is split into smaller overlapping chunks using `RecursiveCharacterTextSplitter`. This splitter tries to break text at natural boundaries (paragraphs, then lines, then sentences, then words) rather than cutting arbitrarily.

```
1 text_splitter = RecursiveCharacterTextSplitter(
2     chunk_size=1000,
3     chunk_overlap=200
4 )
5 chunks = text_splitter.split_documents(documents)
6 print(f"Split into {len(chunks)} chunks")
```

Chunk size (1000 characters): Small enough to keep each chunk semantically focused, large enough to preserve full paragraphs of dense academic text.

Chunk overlap (200 characters, 20%): Prevents an answer-bearing sentence from being split exactly at a chunk boundary, which would otherwise cause neither resulting chunk to fully answer a question.

4.3 Embedding Generation

Each chunk is converted into a dense vector using a local sentence-embedding model. This numeric vector captures the semantic meaning of the text, enabling retrieval by meaning rather than exact keyword matching.

```
1 embeddings = HuggingFaceEmbeddings(  
2     model_name="sentence-transformers/all-MiniLM-L6-v2"  
3 )
```

4.4 Vector Store and Retriever

All chunk embeddings are stored in a FAISS index, which supports fast nearest-neighbour search even as the number of chunks grows. A retriever wraps this index and returns the Top-K most similar chunks for a given question.

```
1 vector_store = FAISS.from_documents(chunks, embeddings)  
2 retriever = vector_store.as_retriever(search_kwargs={"k": 4})
```

Retrieval depth (Top-K = 4): A middle-ground value – low enough to avoid diluting the context with irrelevant chunks, high enough to capture answers that may span more than one section of the paper.

4.5 Context-Grounded Generation

A small, local, instruction-tuned LLM (Qwen2.5-0.5B-Instruct) generates the final answer. It is given a strict prompt that forbids it from using outside knowledge and instructs it to explicitly state when the answer is not present in the retrieved context, which reduces hallucination.

```
1 prompt = f"""You are a research paper question answering  
2     assistant.  
3  
4 Answer the question using ONLY the information provided in the  
5     context.  
6 Do not use outside knowledge. Do not invent information.  
7 If the answer is not available in the context, say:  
8 I could not find this information in the paper.  
9 Give a short and clear answer.  
10  
11 CONTEXT:  
12 {context}  
13  
14 QUESTION:  
15 {question}
```

15 **ANSWER: " "**

4.6 Source Citation

Every retrieved chunk carries page-number metadata from the original PDF. After generating the answer, the system lists the distinct source pages used, allowing the user to verify the answer against the original document.

```

1 used_pages = set()
2 for document in results:
3     page = document.metadata.get("page", "Unknown")
4     page_display = (page + 1) if page != "Unknown" else "Unknown"
5     if page_display not in used_pages:
6         print(f"- {pdf_file.name}, Page {page_display}")
7         used_pages.add(page_display)

```

5 Results

The system was tested on the research paper *"Institutional Distribution in Computer Science Research in India: A Study"*. The document ingestion produced the following statistics:

Metric	Value
Number of pages loaded	6
Number of chunks generated	20
Chunk size	1000 characters
Chunk overlap	200 characters
Retrieval depth (Top-K)	4

The system was then used to automatically answer the five standard research-paper analysis questions (Objective, Methodology, Datasets, Findings, Limitations), each returned with its supporting source page numbers. It also supports an interactive loop for free-form follow-up questions from the user.

6 Viva Voce – Key Discussion Points

Why RAG instead of directly asking an LLM?

An LLM's knowledge is limited to its training data and does not include the specific uploaded document. Asking it directly about the paper's contents leads to either a refusal or a hallucinated answer. RAG instead retrieves the actual relevant passages from the document and forces the model to answer strictly from that retrieved context, grounding every answer in verifiable source text and allowing citation of exact pages.

Why this chunk size?

A chunk size of 1000 characters keeps each chunk small enough to remain semantically focused (so its embedding accurately represents one idea) while still being large enough to preserve full sentences and paragraph context from dense academic text.

Why this overlap?

An overlap of 200 characters (20% of the chunk size) ensures that a sentence or fact sitting near a chunk boundary is not split across two chunks in a way that neither chunk alone can answer a question about it.

Why this retrieval depth (Top-K)?

Top-K = 4 balances two failure modes: too low a value risks missing part of an answer that spans multiple sections of the paper (e.g., limitations mentioned in both Discussion and Conclusion); too high a value floods the LLM's context with irrelevant chunks, which can dilute or confuse the generated answer.

7 Conclusion

This project successfully implements a complete, offline Retrieval-Augmented Generation pipeline for research paper question answering. By combining PDF ingestion, semantic chunking, local embeddings, FAISS-based retrieval, and grounded generation with source citation, the system produces reliable, verifiable answers to both standard research-paper questions and free-form follow-up queries, without requiring any external API key.

References

1. LangChain Documentation, <https://python.langchain.com/>
2. FAISS: A Library for Efficient Similarity Search, Facebook AI Research.
3. Reimers, N., Gurevych, I. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*.
4. Qwen2.5 Technical Report, Alibaba Cloud.
5. Das, A. K., Karanjai, A. *Institutional Distribution in Computer Science Research in India: A Study*, Annals of Library and Information Studies, 2002.